10. Implement All Pair Shortest paths problem using Floyd's algorithm.

```
#include <stdio.h>
#define INF 99999

int main () {
    int n;
    printf("Enter number of vertices:");
    scanf("%d", &n);

    int graph[n][n], dist [n][n];

    printf("Enter the adjacency matrix:\n");
    printf(" Enter %d for INF if no direct edge exists\n",
        INF);

    for(int i = 0; i<n;i++){
        for (int j=0;j<n;j++){
            scanf("%d", &graph[i][j]);
        }
    }

    for (int i = 0; i<n;i++){
        for (int j=0; j<n;j++){
            dist[i][j]= graph[i][j];
        }
    }

    for (int k=0; k<n;k++){
        for(int i=0; i<n; i++){
```

```
for(int j=0;j<n;j++){
    if (dist[i][k] + dist[k][j] < dist[i][j]){
        dist[i][j] = dist[i][k] + dist[k][j]; }
      }
    }
  }
}

printf("\n Shortest distance matrix:\n");
 for (int i =0; i<n; i++){
   for (int j=0;j<n;j++){
      if (dist[i][j]==INF)
          printf("INF");
      else
          printf("%d", dist[i][j]);
   }
   printf("\n");
 }
 return 0;
}
```

output:

Enter number of vertices: 5

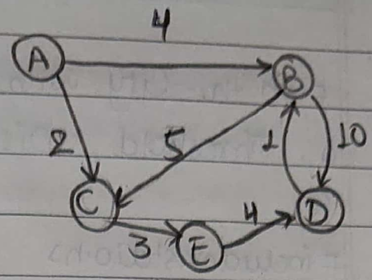
Enter the adjacency matrix:

(Enter 999 for INF if no direct edge exists)

```
0   4   2   999 999
999 0   5   10  999
999 999 0   999 3
999 1   999 0   999
999 999 999 4   0
```

Shortest distance matrix:

0	4	2	9	5
INF	0	5	10	8
INF	8	0	7	3
INF	1	6	0	9
INF	5	10	4	0



Vertices: A, B, C, D, E.

$$D^A = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{bmatrix} 0 & 4 & 2 & \infty & \infty \\ \infty & 0 & 5 & 10 & \infty \\ \infty & \infty & 0 & \infty & 3 \\ \infty & 1 & \infty & 0 & \infty \\ \infty & \infty & \infty & 4 & 0 \end{bmatrix} \end{array}$$

$$D^D = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{bmatrix} 0 & 4 & 2 & 9 & 5 \\ \infty & 0 & 5 & 10 & 8 \\ \infty & 8 & 0 & 7 & 3 \\ \infty & 1 & 6 & 0 & \infty \\ \infty & 5 & \infty & 4 & 0 \end{bmatrix} \end{array}$$

$$D^B = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{bmatrix} 0 & 4 & 2 & \infty & \infty \\ \infty & 0 & 5 & 10 & \infty \\ \infty & \infty & 0 & \infty & 3 \\ \infty & 1 & 6 & 0 & \infty \\ \infty & \infty & \infty & 4 & 0 \end{bmatrix} \end{array}$$

$$D^E = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{bmatrix} 0 & 4 & 2 & 9 & 5 \\ \infty & 0 & 5 & 10 & 8 \\ \infty & 8 & 0 & 7 & 3 \\ \infty & 1 & 6 & 0 & 9 \\ \infty & 5 & 10 & 4 & 0 \end{bmatrix} \end{array}$$

$$D^C = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{bmatrix} 0 & 4 & 2 & \infty & 5 \\ \infty & 0 & 5 & 10 & \infty \\ \infty & \infty & 0 & \infty & 3 \\ \infty & 1 & 6 & 0 & \infty \\ \infty & \infty & \infty & 4 & 0 \end{bmatrix} \end{array}$$

Leetcode Question

1334. Find the City With the Smallest Number of Neighbours at a Threshold Distance

```
#include<stdio.h>
#define INF 1000000

int findTheCity(int n, int** edges, int edgeSize, int* edgesColsize
int distanceThreshold){
        int dist[100][100];

    for(int i=0; i<n ;i++){
        for(int j=0; j<n; j++){
            if(i==j)
                dist[i][j]= 0;

            else
                dist[i][j]= INF;
        }
    }


for(int i=0; i<edgesSize; i++){
    int u= edges[i][0];
    int v=edges [i][1];
    int w =edges [i] [2];

    dist [u][v]=w;
    dist [v][u]= w;
}
```

Bafna Gold

```
for(int k=0;k<n;k++){
    for(int i=0;i<n;i++){
        for(int j=0;j<n;j++){
            if(dist[i][k]+ dist[k][j]<dist[i][j]){
                dist[i][j]=dist[i][k]+dist[k][j];
            }
        }
    }
}

int minCount = INF;
int resultCity = -1;

for(int i=0;i<n;i++){
    int count =0;

    for(int j=0;j<n;j++){
        if(i!=j && dist[i][j]<=distanceThreshold){
            count++;
        }
    }
    if(count <= minCount){
        minCount = count;
        resultCity = i;
    }
}
return resultCity;
}
```

Scanned with OKEN Scanner